Decomposing Algebraic Numbers

into Low-Degree Roots

Exact Mathematica algorithms, global additive optimization, and certified multiplicative decompositions

A constructive study with Wolfram Language code

September 6, 2026

Abstract

Given an exact algebraic number, how can one express it as a sum or product of algebraic numbers while minimizing the largest absolute degree of a component? This article develops a field-theoretic answer and a Wolfram Language implementation. Two-component decompositions in a specified number field reduce to rational linear algebra: addition tests membership in a sum of subfields, whereas multiplication tests a nonzero intersection $F \cap \alpha E$. We construct the complete embedded subfield lattice by polynomial factorization and linear equations, without guessing the component polynomials. For arbitrary finite sums, a normalized-trace argument reduces the unrestricted problem to the normal closure of the input field, giving a complete finite algorithm. For products we provide complete two-component optimization inside an ambient field, exact resultant-based discovery, and exhaustive bounded searches for any specified number of factors; these scopes are explicitly distinguished from unrestricted global optimization. Both ninth-degree examples in the question are decomposed into the requested cubic roots, and degree three is proved globally optimal even when the number of components is unrestricted. The archive contains the complete Wolfram Language package, a local test suite, and independently executed exact-arithmetic validation.

Scope and validation. The mathematical solution for the two supplied examples is complete. The global finite-sum algorithm is proved complete. No general terminating optimizer for unrestricted finite products in all of $\overline{\mathbb{Q}}$ is claimed. The Wolfram connector returned an HTTP 404 in this session, so the supplied Wolfram Language code was not executed in a Mathematica kernel. The algebraic identities and the principal-subfield/pair algorithms were independently checked with exact SymPy arithmetic; a Mathematica test suite is included for local execution.

Contents

1 The answer for the two examples

Let

$$p(X) = X^3 + X + 1, \qquad q(X) = X^3 - X + 1,$$
$$a = \texttt{Root[1+\#+\#\^{}3\&,1]}, \qquad b = \texttt{Root[1-\#+\#\^{}3\&,1]}.$$

Both cubics have a unique real root. Numerically, for identification only,

$$a \approx -0.68232780382801932737, \qquad b \approx -1.32471795724474602596.$$

Write the input polynomials as

$$f_\times(X) = X^9 + 2X^7 - 3X^6 + X^5 - X^4 + 3X^3 - X - 1, \tag{1}$$
$$f_+(X) = X^9 + 6X^6 + 3X^5 - 15X^3 + 24X^2 - 4X + 8. \tag{2}$$

The requested answers are exactly

$$\boxed{\alpha_\times = ab}, \qquad \boxed{\alpha_+ = a + b}.$$

Their maximum component degree is *three*, and no finite sum or product of degree-one and degree-two numbers can represent either input. This is a global minimality statement, not merely a successful search among pairs of cubics.

1.1 Immediate exact checks in Mathematica

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];
alphaProduct = Root[
  -1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 + 2 #^7 + #^9 &, 1];
alphaSum = Root[
  8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 + 6 #^6 + #^9 &, 1];

RootReduce[alphaProduct - a b]      (* 0 *)
RootReduce[alphaSum - a - b]        (* 0 *)
Exponent[MinimalPolynomial[#, x], x] & /@ {a, b}
(* {3, 3} *)
```

These comments give the mathematically established expected outputs, not a transcript from an available Mathematica kernel. `RootReduce` canonicalizes algebraic combinations, and `MinimalPolynomial` computes the absolute polynomial over $\mathbb{Q}$ in the unqualified form used here [4, 5].

1.2 Automatic discovery, without entering the cubics

Load the accompanying package and run:

```
Get["code/RootDecomposition.wl"];

productData = RootSubfieldData[alphaProduct];
sumData = RootSubfieldData[alphaSum];

productAnswer = RootPairDecompose[
  alphaProduct, "Product", productData];
sumAnswer = RootPairDecompose[alphaSum, "Sum", sumData];

productAnswer["Components"]
sumAnswer["Components"]
```

The algorithm obtains the cubic subfields from the input alone. It does not use a coefficient-height bound, an integer-relation guess, or a list of candidate cubic polynomials. Rational normalization of the two components removes the arbitrary rational scaling in the product and the arbitrary rational translation in the sum. The components are the requested a, b, possibly in the opposite order. The independent reference implementation, before that optional normalization, found rationally rescaled cubic factors, as expected.

For both inputs the complete subfield-degree list is

$$\{1, 3, 3, 9\},$$

and the input polynomial factors over its own number field into irreducible factors of degrees

$$\{1, 2, 2, 4\}.$$

The exact independent checks found component degrees $\{3, 3\}$ and verified the identities. The expected result fields include

```
"MaximumDegree" -> 3
"Verified" -> True
"GlobalDegreeLowerBound" -> 3
"GlobalOptimalityProved" -> True
```

The last field is justified by a theorem below, not by exhausting only a finite coefficient search.

2 What exactly is being minimized?

For $\beta \in \overline{\mathbb{Q}}$, define

$$\deg_{\mathbb{Q}}(\beta) = [\mathbb{Q}(\beta) : \mathbb{Q}].$$

The intrinsic degree is the degree of its minimal polynomial, not the degree of an arbitrarily padded defining polynomial and not a relative degree over an algebraic coefficient field. We assume that every polynomial underlying a component `Root` has rational coefficients. Allowing arbitrary algebraic coefficients would trivialize the question: $X - \alpha$ has degree one over $\mathbb{Q}(\alpha)$.

For a finite expression

$$\alpha = \beta_1 + \cdots + \beta_r \quad \text{or} \quad \alpha = \beta_1 \cdots \beta_r,$$

its cost is $\max_i \deg_{\mathbb{Q}}(\beta_i)$. Nonzero factors are assumed when $\alpha \neq 0$. Rationals have degree one; zero is handled separately in the product routine. Adding zero, multiplying by one, or moving a rational constant between two components does not affect the minimum. Complex algebraic components are permitted unless a real-only restriction is explicitly imposed.

There are three independent distinctions.

1. **Two components versus arbitrarily many.** Minimizing over pairs is a different optimization problem from minimizing over all finite expressions.
2. **Internal versus unrestricted.** Requiring every component to lie in $K = \mathbb{Q}(\alpha)$ can change the answer. The input does not impose that requirement.
3. **Discovery versus proof of optimality.** An exact identity proves an upper bound. A lower bound or a complete exhaustion theorem is needed to prove a minimum.

A responsible implementation must make these distinctions visible in its output.

2.1 Why two components can be insufficient

For

$$u = \sqrt{2} + \sqrt{3} + \sqrt{5},$$

the unrestricted additive minimum is two, achieved by three summands. Its degree is eight: the eight independent sign changes give eight distinct conjugates. Two degree-at-most-two summands could generate a field of degree at most four, so a pair cannot achieve the same cost. Similarly,

$$v = (1+\sqrt{2})(1+\sqrt{3})(1+\sqrt{5})$$

has degree eight and is a product of three quadratic factors. To see its degree, expand in the standard basis of $\mathbb{Q}(\sqrt{2},\sqrt{3},\sqrt{5})$; every basis coefficient is nonzero, so no nonidentity sign change fixes it. Two quadratic factors again cannot suffice. Consequently, a complete pair optimizer is not automatically a complete optimizer for finite products.

3 Universal lower bounds and global certificates

Let $P(n)$ denote the largest prime divisor of $n > 1$, with $P(1) = 1$.

Theorem 3.1 (Prime-divisor obstruction)**.** *If an algebraic number of degree n is a finite sum or product of numbers of degrees at most d, then $d \geq P(n)$. The same obstruction holds for any expression formed from such numbers by rational field operations.*

Proof. For each component β_i of degree $m_i \leq d$, take the normal closure L_i of $\mathbb{Q}(\beta_i)$. Its Galois group embeds in S_{m_i}, so every prime dividing $[L_i : \mathbb{Q}]$ is at most d. The compositum M of the finitely many L_i is Galois, and restriction embeds $\mathrm{Gal}(M/\mathbb{Q})$ in $\prod_i \mathrm{Gal}(L_i/\mathbb{Q})$. Thus every prime dividing $[M : \mathbb{Q}]$ is at most d. Since $\mathbb{Q}(\alpha) \subseteq M$, the tower formula gives $n \mid [M : \mathbb{Q}]$, which proves the assertion. □

For the examples, $n = 9$ and $P(n) = 3$. The cubic decompositions therefore attain a universal lower bound. Equivalently, any finite compositum of quadratic fields has power-of-two degree, whereas a degree-nine field cannot be its subfield.

Corollary 3.2. *An algebraic number of prime degree p cannot be expressed as a finite sum or product of numbers all of degree less than p.*

For exactly two components there is the additional elementary bound

$$\deg_{\mathbb{Q}}(\alpha) \leq [\mathbb{Q}(\beta,\gamma) : \mathbb{Q}] \leq \deg_{\mathbb{Q}}(\beta)\deg_{\mathbb{Q}}(\gamma),$$

so $d \geq \lceil \sqrt{n} \rceil$. This latter bound must not be applied unchanged to arbitrarily many components.

Remark 3.3. In general $[\mathbb{Q}(\beta,\gamma) : \mathbb{Q}]$ need not divide $\deg_{\mathbb{Q}}(\beta)\deg_{\mathbb{Q}}(\gamma)$. For example, two distinct conjugate cubic root fields can have a compositum of degree six. The inequality is valid; the naive divisibility assertion is not. The proof of Theorem 3.1 deliberately uses normal closures.

3.1 A stronger Galois-group obstruction

Let L be the normal closure of $\mathbb{Q}(\alpha)$ and let $G = \mathrm{Gal}(L/\mathbb{Q})$. If all components have degree at most d, then

$$\exp(G) \mid \mathrm{lcm}(1, 2, \ldots, d). \tag{3}$$

Indeed, the group of the common normal closure embeds in a product of symmetric groups of degrees at most d, and G is a quotient of that group. Subgroups and quotients cannot increase the exponent beyond this least common multiple. This is a necessary condition, not a sufficient one. It will distinguish an internal minimum from a global minimum in Section 9.

4 Resultants: exact discovery with clearly stated limits

The existing Mathematica Stack Exchange answer constructs companion matrices for guessed component polynomials, then compares a Kronecker-product or Kronecker-sum characteristic polynomial with the input [1]. Resultants give the same elimination polynomials more directly; `Resultant` is the appropriate Wolfram Language primitive [9].

If p, q are monic of degrees m, n, define

$$C_+(p,q)(X) = \mathrm{Res}_Y\big(p(Y), q(X-Y)\big), \tag{4}$$
$$C_\times(p,q)(X) = \mathrm{Res}_Y\big(p(Y), Y^n q(X/Y)\big). \tag{5}$$

The second argument in the product formula is a polynomial after expansion. These resultants are products over all pairs of conjugates:

$$C_+(p,q) = \prod_{i,j}(X - a_i - b_j), \qquad C_\times(p,q) = \prod_{i,j}(X - a_i b_j).$$

They are monic of degree mn, including multiplicities.

For the present cubics,

$$\mathrm{Res}_Y(Y^3 + Y + 1, (X-Y)^3 - (X-Y) + 1) = f_+(X), \tag{6}$$
$$\mathrm{Res}_Y(Y^3 + Y + 1, X^3 - XY^2 + Y^3) = f_\times(X). \tag{7}$$

Both equalities were checked by exact polynomial arithmetic.

4.1 The divisibility condition, not an unconditional equality

If the input minimal polynomial is f and a proposed pair has minimal polynomials p, q, the necessary relation is

$$f \mid C_+(p,q) \quad \text{or} \quad f \mid C_\times(p,q).$$

Equality requires additional degree information. For example,

$$C_+(X^2 - 2, X^2 - 2) = X^2(X^2 - 8),$$

whereas the minimal polynomial of $2\sqrt{2}$ is only $X^2 - 8$. Equating a degree-four resultant to this degree-two target would reject a genuine decomposition.

There is a second pitfall: solving the coefficient equations over $\mathbb{C}$ can return algebraic coefficients for p and q. That does not solve the original rational-coefficient problem. General symbolic equation solving is not, merely by its name, a complete decision method for rational points in the coefficient space. A successful rational specialization must be checked, and failure to find one is not a proof of nonexistence.

4.2 An effective inverse-resultant method

A useful practical compromise is to guess or enumerate *one* candidate polynomial p and derive the possible partner polynomials automatically. If f is the input minimal polynomial, use

$$R_+(X) = \mathrm{Res}_Y\big(p(Y), f(X+Y)\big), \tag{8}$$
$$R_\div(X) = \mathrm{Res}_Y\big(p(Y), f(XY)\big). \tag{9}$$

If $\alpha = \beta + \gamma$ or $\alpha = \beta\gamma$ and $p(\beta) = 0$, then γ is a zero of the corresponding resultant. Factor it over $\mathbb{Q}$, retain factors of degree at most the target bound, and check their roots against the *specified* input root. The package exposes this as

```
RootPairFromPolynomial[alphaProduct, x^3+x+1, x, "Product", 3]
RootPairFromPolynomial[alphaSum,     x^3+x+1, x, "Sum",     3]
```

This returns an exact certificate when successful. It is complete for the chosen candidate polynomial and partner-degree bound, but it does not prove that the chosen candidate polynomial was the only possible one.

5 Explicit rational recovery formulas for the examples

The components actually belong to the fields generated by their sum and product. More strongly, they can be recovered by short rational functions.

5.1 Recovering the factors from the product

Set $t = ab$. Substituting $b = t/a$ into $b^3 - b + 1 = 0$, then using $a^3 + a + 1 = 0$, gives

$$ta^2 + a + 1 - t^3 = 0.$$

Eliminating a^2 between this relation and the cubic yields

$$(t^4 + t^2 - t + 1)a - t^3 + t^2 + 1 = 0.$$

Hence

$$\boxed{a = \frac{t^3 - t^2 - 1}{t^4 + t^2 - t + 1}, \qquad b = \frac{t}{a}.} \tag{10}$$

The denominator is positive for real t, since

$$t^4 + t^2 - t + 1 = t^4 + (t - \tfrac{1}{2})^2 + \tfrac{3}{4} > 0.$$

Thus this is valid for the real product in the question, with no choice of a radical branch.

5.2 Recovering the summands from the sum

Set $s = a + b$ and eliminate b from the two cubics. The linear subresultant in a is

$$(6s^4 - 6s + 4)a - 3s^5 + 5s^3 + 3s^2 - 2s + 4.$$

Consequently,

$$\boxed{a = \frac{3s^5 - 5s^3 - 3s^2 + 2s - 4}{6s^4 - 6s + 4}, \qquad b = s - a.} \tag{11}$$

Again there is no real denominator problem:

$$6s^4 - 6s + 4 = 6(s^2 - \tfrac{1}{2})^2 + 6(s - \tfrac{1}{2})^2 + 1 > 0.$$

Here is a particularly small implementation specialized to the two examples:

```
productParts[t_] := Module[{u},
  u = RootReduce[(t^3-t^2-1)/(t^4+t^2-t+1)];
  {u, RootReduce[t/u]}
];
sumParts[s_] := Module[{u},
  u = RootReduce[(3s^5-5s^3-3s^2+2s-4)/(6s^4-6s+4)];
```

```
  {u, RootReduce[s-u]}
];
productParts[alphaProduct]
sumParts[alphaSum]
(* Each returns the two requested cubic roots. *)
```

These formulas are explanatory certificates for this instance. The general subfield algorithm does not assume them.

5.3 Why the input degrees really are nine

The cubics p and q are irreducible by the rational-root test. Their discriminants are -31 and -23, respectively, so their splitting fields have Galois group S_3 and different quadratic subfields $\mathbb{Q}(\sqrt{-31})$ and $\mathbb{Q}(\sqrt{-23})$.

The intersection of the two splitting fields is Galois over $\mathbb{Q}$. A common nontrivial quotient of two S_3 groups could be C_2 or S_3; either possibility would force the quadratic subfields to agree. Therefore the intersection is $\mathbb{Q}$, and the joint Galois group is $S_3 \times S_3$. Its orbit on the ordered pair (a, b) has size $3 \cdot 3 = 9$, proving

$$[\mathbb{Q}(a, b) : \mathbb{Q}] = 9.$$

The recovery formulas show $\mathbb{Q}(ab) = \mathbb{Q}(a + b) = \mathbb{Q}(a, b)$. Thus the degree-nine resultants are the minimal polynomials, rather than extraneous multiples.

Each cubic has only one real root because its discriminant is negative. Every real root of either degree-nine polynomial recovers real roots of both cubics by the displayed rational formulas, so the degree-nine polynomials also have only one real root. Wolfram's indexing convention places real roots first in increasing order [3]. This proves that index 1 is precisely the requested root in both inputs. Numerically,

$$ab \approx 0.90389189445834756110, \qquad a + b \approx -2.00704576107276535333.$$

6 The central reduction: two components in a fixed field

Let $K/\mathbb{Q}$ be a finite number field containing the specified α, with a fixed embedding into $\mathbb{C}$. Represent K as a rational vector space. For embedded subfields $F, E \subseteq K$, choose rational bases and write their coordinate vectors as the rows of matrices B_F, B_E.

Theorem 6.1 (Additive pair test)**.** *There are $\beta \in F$ and $\gamma \in E$ with $\alpha = \beta + \gamma$ if and only if*

$$\alpha \in F + E.$$

In coordinates, this is the rational linear system

$$[\, B_F^{\mathsf{T}} \; B_E^{\mathsf{T}} \,] \begin{bmatrix} u \\ v \end{bmatrix} = [\alpha]. \tag{12}$$

A solution gives $\beta = u^{\mathsf{T}} B_F$ and $\gamma = v^{\mathsf{T}} B_E$, interpreted as field elements.

Proof. A vector is in the sum of two vector spaces precisely when it is a sum of vectors in those spaces. The displayed matrix is the coordinate form of this assertion. □

The product case is almost as simple, but the useful reformulation is less obvious.

Theorem 6.2 (Multiplicative pair test)**.** *For $\alpha \neq 0$, the following are equivalent:*

$$\alpha \in F^{\times}E^{\times} \quad \Longleftrightarrow \quad F \cap \alpha E \neq \{0\}.$$

Let M_α be the rational matrix of multiplication by α on K. A nonzero vector in the nullspace of

$$[\, B_F^{\mathsf{T}} \; - M_\alpha B_E^{\mathsf{T}} \,] \tag{13}$$

gives $u \in F$, $v \in E$ satisfying $u = \alpha v$, and hence the decomposition $\alpha = u\, v^{-1}$.

Proof. If $\alpha = \beta\gamma$, then $\beta = \alpha\gamma^{-1}$ is a nonzero member of $F \cap \alpha E$. Conversely, a nonzero $u = \alpha v$ in that intersection has $v \neq 0$, and $u \in F$, $v^{-1} \in E$ give the required factors. A nonzero nullspace vector in (13) cannot have $v = 0$, because the columns of each basis matrix are independent in their respective coordinate maps and multiplication by α is invertible. □

This test avoids logarithms, unit-group generators, ideal factorization and rational-point searches. Those are unnecessary for the two-subfield product problem.

6.1 Optimizing the degree among pairs

Compute all embedded subfields $F \subseteq K$. Order all unordered pairs (F, E) by

$$\max([F : \mathbb{Q}], [E : \mathbb{Q}]).$$

Apply the appropriate linear test and stop at the first success. Every valid component $\beta \in K$ lies in its own generated subfield $\mathbb{Q}(\beta)$, whose dimension equals its absolute degree. Therefore enumerating *all* subfields cannot miss a pair with smaller maximum degree. This proves completeness and optimality for pairs in K.

A returned element may incidentally have smaller degree than its containing field, but that creates no gap in the proof: its smaller generated field is also in the enumeration. The routine remeasures the actual component degrees using `MinimalPolynomial` before issuing its certificate.

6.2 Rational normalization

Additive pairs admit the replacement $(\beta, \gamma) \mapsto (\beta - c, \gamma + c)$ for $c \in \mathbb{Q}$. Centering the first component by its mean conjugate removes this ambiguity in the linearly disjoint cubic example. Multiplicative pairs admit $(\beta, \gamma) \mapsto (\beta/c, c\gamma)$ for $c \in \mathbb{Q}^{\times}$. When the monic minimal polynomial's constant term has an appropriate rational root, the package uses it to remove a rational scale. These normalizations change neither the identity nor the component degrees.

7 Computing every embedded subfield in Mathematica

There is no need to assume an unspecified `Subfields` routine. The included implementation constructs the subfields explicitly. Its organizing principle is the principal-subfield method: factor a defining polynomial over its own field, form certain linear equalizers, and close them under intersections. Principal subfields and efficient intersection algorithms are studied by Szutkoski and van Hoeij [2]. We give the needed correctness argument here.

Let $K = \mathbb{Q}(\theta)$ have degree N, and let $f \in \mathbb{Q}[X]$ be the monic minimal polynomial of θ. For each monic factor $g \mid f$ over K, put

$$V_g = \{h(\theta) : h \in \mathbb{Q}[X],\ \deg h < N,\ g(X) \mid h(X) - h(\theta)\}. \tag{14}$$

Lemma 7.1. *V_g is a subfield of K. If g_1, g_2 are coprime factors of f, then*

$$V_{g_1 g_2} = V_{g_1} \cap V_{g_2}.$$

Proof. Closure under rational linear combinations follows immediately from the congruence in (14). For multiplication, multiply the two congruences and reduce the product polynomial modulo f; because $g \mid f$, this reduction does not change the congruence modulo g.

For inversion, suppose $h(\theta) \neq 0$. There is $k \in \mathbb{Q}[X]$ with $h(X)k(X) \equiv 1 \pmod f$, since f is irreducible and does not divide h. Modulo g, the congruence $h(X) \equiv h(\theta)$ implies $k(X) \equiv h(\theta)^{-1}$. Evaluating the identity modulo f at θ shows $k(\theta) = h(\theta)^{-1}$. Thus the inverse lies in V_g. Finally, divisibility by a product of coprime polynomials is equivalent to divisibility by each factor. □

Lemma 7.2. *Every intermediate field F is V_g for $g = \mathrm{minpoly}_F(\theta)$.*

Proof. If $h(\theta) \in F$, then $h(X) - h(\theta) \in F[X]$ vanishes at θ, so g divides it. This gives $F \subseteq V_g$. Conversely, if $h(\theta) \in V_g$, all conjugates of θ over F have the same value under h. Since the extension is separable,

$$\mathrm{Tr}_{K/F}(h(\theta)) = [K : F]h(\theta).$$

The left side belongs to F, and division by $[K : F]$ is allowed in characteristic zero. Hence $h(\theta) \in F$. □

Factor $f = g_1 \cdots g_r$ into irreducibles over K. The two lemmas show that every subfield is an intersection of some of the V_{g_i}. Starting from K and successively intersecting each existing subspace with each V_{g_i} produces the complete field lattice; canonical reduced row echelon forms remove duplicates.

7.1 The exact matrix for a principal subfield

Write $h(X) = \sum_{j=0}^{N-1} c_j X^j$ with unknown rational c_j. Compute

$$R_j(X) = \mathrm{rem}(X^j, g) - \theta^j.$$

Expand each coefficient of R_j in the basis $1, \theta, \ldots, \theta^{N-1}$. Stack those rational coordinate vectors into a column. If A_g is the matrix of these columns, then

$$V_g \quad \text{has row basis} \quad \mathrm{NullSpace}(A_g).$$

There are at most $N \deg g$ scalar equations in N unknowns. The implementation uses exact `PolynomialRemainder` and rational `RowReduce`/`NullSpace` operations; it never samples numerical conjugates [10].

7.2 Coordinate discipline matters

The coordinate system must use a fixed generator. `ToNumberField` and `AlgebraicNumber` can normalize a nonintegral generator to an algebraic integer [6, 7]. To avoid silently mixing coordinate systems, the package first replaces an input generator η by $\theta = c\eta$, where c is the leading coefficient of its primitive integer minimal polynomial. Then θ is integral and $\mathbb{Q}(\theta) = \mathbb{Q}(\eta)$.

For example, if

$$cX^N + c_{N-1}X^{N-1} + \cdots + c_0 = 0,$$

then $\theta = c\eta$ satisfies the monic integer polynomial

$$X^N + c_{N-1}X^{N-1} + cc_{N-2}X^{N-2} + \cdots + c^{N-1}c_0.$$

The implementation still checks that `ToNumberField` returned the intended generator and that all coordinate entries are rational. `AlgebraicNumberPolynomial` extracts the polynomial coordinate representation [8]. A power basis is sufficient; computing an integral basis is not required.

8 A complete global algorithm for arbitrary finite sums

In an ambient field K, define

$$W_d(K) = \sum_{\substack{F \subseteq K \\ [F:\mathbb{Q}] \le d}} F.$$

This is a *vector-space sum*, not a compositum of fields.

Theorem 8.1 (Finite sums in an ambient field)**.** *An element $\alpha \in K$ is a finite sum of elements of degrees at most d lying in K if and only if $\alpha \in W_d(K)$.*

Proof. Each eligible summand belongs to its generated subfield of degree at most d. Conversely, membership in the vector-space sum expresses α as a sum of elements of the eligible fields; each such element has degree at most the field's degree. □

Thus all finite sums in K can be optimized by concatenating subfield bases and solving a rational linear system. Rational coefficients are absorbed into the summands. There is no need to impose a bound on the number of summands. The deterministic solver sets free variables to zero, so at most $[K : \mathbb{Q}]$ basis coefficients need be nonzero; grouping by subfield gives at most that many nonzero terms.

The important question is whether using $K = \mathbb{Q}(\alpha)$ is enough. In general it is not. The following descent theorem gives a correct global replacement.

Theorem 8.2 (Normalized-trace descent)**.** *Let $L/\mathbb{Q}$ be a finite Galois extension and $\alpha \in L$. If α is a finite sum of algebraic numbers of degrees at most d, then it is a finite sum of elements of L of degrees at most d. The number of nonzero summands need not increase.*

Proof. Take a common finite Galois extension $M/\mathbb{Q}$ containing L and all the summands. Put $H = \mathrm{Gal}(M/L)$, which is normal in $\mathrm{Gal}(M/\mathbb{Q})$, and define

$$P_L(x) = \frac{1}{|H|} \sum_{h \in H} h(x).$$

This is a rational-linear projection onto L, fixing α. Normality of H implies

$$P_L(\sigma x) = \sigma P_L(x) \qquad (\sigma \in \mathrm{Gal}(M/\mathbb{Q})).$$

Therefore every automorphism fixing x also fixes $P_L(x)$. By the Galois correspondence,

$$P_L(x) \in \mathbb{Q}(x), \qquad \deg_{\mathbb{Q}}(P_L(x)) \mid \deg_{\mathbb{Q}}(x).$$

In particular $P_L(x) \in L \cap \mathbb{Q}(x)$ and its degree does not increase. Apply P_L term by term to the original sum and discard any zero terms. □

Corollary 8.3 (Global additive optimum)**.** *If L is the normal closure of $\mathbb{Q}(\alpha)$, then the unrestricted minimum for finite sums is*

$$\boxed{\min\{d : \alpha \in W_d(L)\}.}$$

This minimum is computable by a finite exact algorithm.

The package first tries $\mathbb{Q}(\alpha)$. If the result reaches the universal lower bound, it is already globally optimal and no normal closure is needed. Otherwise `RootGlobalSumDecompose` constructs a primitive element of the field generated by all conjugates, computes its subfields, and applies the same vector-space procedure. It deliberately uses `ToNumberField[...,All]`, because the `Automatic` setting need not use the smallest common extension [6].

```
RootSumDecompose[alphaSum]       (* Complete in Q(alphaSum). *)
RootGlobalSumDecompose[alphaSum] (* Complete globally for sums. *)

RootGlobalSumDecompose[Sqrt[2]+Sqrt[3]+Sqrt[5]]
(* Maximum component degree 2, with a globally valid certificate. *)
```

For a real-only version, one should restrict eligible fields to subfields contained in the real part of the chosen embedding of L. The unrestricted implementation does not silently impose that additional requirement.

8.1 Optional minimization of the number of summands

After finding the smallest d, enumerate subcollections of eligible subfields in increasing cardinality and test whether their vector-space sum contains α. Two summands in the same field can be combined, so this also gives a complete, although potentially expensive, secondary optimization of the number of terms. It is not needed for the requested maximum-degree objective.

9 Why the original field can be too small

An explicit counterexample is instructive. Let $r_1, \dots, r_4$ be the roots of

$$X^4 - X - 1,$$

and let $\alpha = r_1 + r_2$ be a sum of two distinct roots. The six pair sums have polynomial

$$f(X) = X^6 + 4X^2 - 1.$$

For example, the sum of the two real quartic roots is the positive real root of this sextic.

The quartic has Galois group S_4: modulo 2 it is irreducible, giving a four-cycle, and modulo 7 it has one linear and one irreducible cubic factor, giving a three-cycle. The transitive subgroup of S_4 with these cycle types is S_4. The stabilizer of a specified two-element subset is

$$H = \langle (12), (34) \rangle,$$

of order four, so the pair sum has degree six. The only proper nontrivial overgroup of H is the order-eight stabilizer of the associated unordered partition. Thus $\mathbb{Q}(\alpha)$ has subfield degrees exactly

$$1, 3, 6,$$

with unique nontrivial proper subfield $\mathbb{Q}(\alpha^2)$. Any sum of elements from its proper subfields stays in that cubic field, and cannot equal α. The minimum for finite sums *inside* $\mathbb{Q}(\alpha)$ is therefore six.

But $\alpha = r_1 + r_2$ is a sum of two degree-four numbers outside that field. Its normal closure has group S_4: from all the pair sums one recovers each quartic root, for example

$$r_1 = \tfrac{1}{2}\big((r_1 + r_2) + (r_1 + r_3) - (r_2 + r_3)\big).$$

Since S_4 contains elements of order four, obstruction (3) rules out all expressions built from degree-at-most-three numbers. Consequently the unrestricted additive minimum is exactly four, strictly less than the internal minimum six.

The independent subfield implementation also found the degree list $\{1, 3, 6\}$ for this sextic. This example is why the package reports the ambient-field scope instead of promoting every internal optimum to a global one.

10 Products beyond two factors: exact search, honest certificates

The two-subfield intersection test is complete and finite. Applying it recursively can be useful, but a greedy choice of one pair decomposition need not lead to a globally optimal many-factor expression. The example of three independent quadratic factors already shows that one must distinguish the two objectives.

10.1 Why the trace proof does not transfer to products

If $\alpha = \prod_i \beta_i$ in a common Galois extension M and L is a normal subfield, taking norms gives

$$\alpha^{[M:L]} = \prod_i \mathrm{N}_{M/L}(\beta_i).$$

This is a relation for a *power* of α, not for α itself. Extracting roots can increase the absolute degrees and introduce roots of unity. Membership of a power in a multiplicative subgroup does not imply membership of the element: $i^2 = -1 \in \mathbb{Q}^\times$, yet $i \notin \mathbb{Q}^\times$. Thus the additive projection proof cannot be cited as a proof that a product optimum is attained in the normal closure.

Likewise, taking numerical logarithms is not a replacement for an exact product test. It loses torsion information, introduces branch choices, and turns exact multiplicative relations into precision-dependent integer-relation guesses. Exact reduction of the final identity is essential.

10.2 A complete search within explicit finite bounds

Define the coefficient height of an algebraic number to be the maximum absolute coefficient of its primitive integer minimal polynomial with positive leading coefficient. For integers $d, H \geq 1$, enumerate all polynomials

$$a_m X^m + \cdots + a_0, \quad 2 \leq m \leq d, \quad 1 \leq a_m \leq H, \quad |a_j| \leq H,$$

with coefficient gcd one, retaining only irreducible polynomials. Include every root of each retained polynomial. This finite catalogue includes nonmonic polynomials, so rational denominators have not been silently excluded.

For a maximum of R factors, enumerate multisets of at most $R - 1$ catalogue elements. For each partial product u, compute the exact remainder factor $\gamma = \alpha/u$ and test $\deg_{\mathbb{Q}}(\gamma) \leq d$. The final factor has no coefficient-height restriction.

Proposition 10.1 (Bounded-product completeness)**.** *The search finds every representation with at most R factors of degrees at most d for which all but at most one nonrational factor have coefficient height at most H.*

Proof. Rational factors can be combined and absorbed into the unrestricted last factor without changing its degree. Order the remaining bounded-height factors nondecreasingly in the catalogue. Their multiset appears in the enumeration. At that step the exact quotient is the remaining factor, whose degree test succeeds. □

```
BoundedRootProduct[alphaProduct, 3, 1, 2]

RootProductFromCandidates[
  (1+Sqrt[2])(1+Sqrt[3])(1+Sqrt[5]),
  {1+Sqrt[2],1+Sqrt[3],1+Sqrt[5]}, 2, 3]
```

Both are mathematically successful instances; the latter explicitly searches a three-factor quadratic decomposition. A successful result always includes an exact identity certificate. When its degree reaches the universal lower bound, it is globally optimal regardless of the search bounds.

A negative result says `"NotFoundWithinBounds"`, not "irreducible" and not "globally optimal." Increasing H and R gives a semidecision procedure for a fixed degree threshold: every existing finite product decomposition is eventually found. It does not provide a terminating nonexistence decision in all negative cases, and it does not by itself compute the global minimum over all of $\overline{\mathbb{Q}}$. This article does not claim otherwise.

10.3 A practical product workflow

First compute an internal two-factor decomposition and the universal lower bound. If they coincide, stop: the optimum is proved. Otherwise use problem-specific candidate polynomials, the inverse-resultant filter, larger ambient fields when justified, and bounded many-factor searches. Retain the best exact upper bound and any separately established lower bounds. Never replace an unresolved gap between those bounds by a claim of minimality.

11 Implementation interface and operational details

The complete package appears in Appendix A; it is also supplied in the archive as `code/RootDecomposition.wl`. The important entry points are summarized here.

Function	Guarantee on successful completion
`RootDegree[a]`	Absolute algebraic degree; inexact input is rejected.
`RootDegreeLowerBound[a]`	Largest prime divisor of the input degree, valid for any finite sum or product.
`RootSubfieldData[theta]`	Complete list of embedded subfields of $\mathbb{Q}(\theta)$, represented by rational row bases.
`RootPairDecompose[a,op,data]`	Optimal maximum degree for two components in the supplied ambient field. Default ambient field: $\mathbb{Q}(a)$.
`RootSumDecompose[a,data]`	Optimal maximum degree for arbitrary finite sums in that ambient field.

Function	Guarantee on successful completion
`RootGlobalSumDecompose[a]`	Unrestricted global additive optimum, by normal-closure descent if needed.
`RootPairFromPolynomial[...]`	Complete partner search for one supplied polynomial and a partner-degree bound, with exact branch verification.
`BoundedRootProduct[a,d,H,R]`	Exhaustive bounded product search with an unrestricted last factor; negative results remain explicitly scoped.

11.1 Exact branches and expressions

A minimal polynomial specifies a conjugacy class, not a particular algebraic number. Every returned expression is therefore checked against the input by

```
TrueQ[RootReduce[expression - input] === 0]
```

The `"Components"` list is retained as well as the combined expression. This avoids losing the useful decomposition if subsequent evaluation or formatting recombines parts of it. Lower-degree `Root` objects can simplify automatically to rationals or radicals; the mathematical component degree is still measured with `MinimalPolynomial`, not by counting the syntactic degrees of visible `Root` heads [3].

11.2 Errors are not mathematical negative answers

The package returns a `Failure` when field conversion, factorization or exact verification does not complete as expected. It does not treat an unevaluated `ToNumberField` call as a zero coordinate vector. The documentation explicitly states that `ToNumberField[a,theta]` can remain unevaluated when a is outside the requested field [6].

For expensive calls, a user can impose a resource limit:

```
TimeConstrained[
  RootGlobalSumDecompose[alpha],
  120,
  Failure["TimeLimit", <|"MathematicalConclusion" -> "None"|>]
]
```

The number here is a user-chosen runtime limit, not an estimate of the routine's running time. A timeout provides no nonexistence result.

11.3 A Wolfram Language subtlety to avoid

The unqualified `MinimalPolynomial[a,x]` is used to measure absolute degree. The documented `Extension` form can instead return a characteristic polynomial that is a power of the relative minimal polynomial [5]. The implementation does not infer relative field degrees by naively taking that polynomial's degree. It obtains them from exact subfield dimensions and polynomial factorization.

12 Cost, limitations, and useful improvements

The algorithms are exact, but not uniformly inexpensive. If the ambient degree is N, principal-subfield matrices have N columns and at most $N \deg g$ rows. Rational coefficient growth can matter as much as matrix dimensions. If there are s subfields, the pair optimizer performs at most $s(s+1)/2$ linear tests. The complete field lattice itself can be large; faster partition-based intersection methods are available in the cited subfield literature [2].

The most important possible blow-up is the normal closure. An input field of degree n can have normal closure of degree as large as $n!$. The two supplied examples have normal closure degree 36, but the input-field computation already attains the lower bound and avoids constructing it.

Before irreducibility filtering, a coefficient catalogue contains at most

$$\sum_{m=2}^{d} H(2H+1)^m$$

polynomials. The number of partial products grows combinatorially with both the catalogue size and the factor-count bound. Therefore the bounded routine is a transparent baseline and certificate finder, not a promise of practical performance at large bounds.

Several improvements preserve correctness. Cache all field-coordinate conversions and multiplication matrices; use modular linear algebra with exact reconstruction; apply the inverse-resultant degree filter before expensive root comparisons; deduplicate identical partial products; replace the simple field-lattice closure by a specialized subfield implementation; and use domain knowledge to propose small candidate polynomials. Every accelerated candidate must still pass the exact identity and absolute-degree checks.

13 Validation and reproducibility

The archive includes three kinds of evidence, deliberately kept separate.

Mathematical proofs. The linear pair tests, completeness of the subfield construction, normalized-trace descent, and lower bounds are proved in the article. The displayed rational recovery formulas give small explicit certificates for the examples.

Executed independent checks. The script `tests/exact_validation.py` uses SymPy exact algebraic-field arithmetic. It recomputes both resultants, verifies irreducibility and the number of real roots, factors each ninth-degree polynomial over its own embedded field, constructs the principal subfields and their intersections, checks that every computed subspace contains one and is closed under multiplication, solves both pair problems, measures component degrees, and verifies the recovered identities and the short rational recovery formulas. It also checks the rational normalization of the returned pairs and the sextic counterexample in Section 9. The recorded results are in `tests/validation_results.json` and `tests/validation_log.txt`. Numerical values are printed only as labels for real roots.

Supplied Mathematica tests. `code/RunTests.wl` contains exact `VerificationTest` checks for the examples, the resultant front end, rational and zero cases, nonintegral inputs, three quadratic summands, and three quadratic factors. These tests were *not executed in Mathematica in this session*: the available Wolfram MCP endpoint returned HTTP 404 for both context lookup and evaluation. This is an implementation-validation limitation, not an unresolved identity in the two examples.

To reproduce locally, from the extracted archive directory:

```
python tests/exact_validation.py
```

```
wolframscript -file code/RunTests.wl
pdflatex -interaction=nonstopmode -halt-on-error root_decomposition.tex
pdflatex -interaction=nonstopmode -halt-on-error root_decomposition.tex
```

The independent Python checks require SymPy. The Wolfram package uses documented built-in functionality and no third-party number-field package. As with any unexecuted implementation port, local tests should be run before relying on it in a larger automated workflow.

14 Conclusions

The useful structural move is to stop treating the problem solely as inverse symbolic polynomial composition. An algebraic number lives in a finite-dimensional rational vector space, and its low-degree components live in subfields. For two components, addition is a span test and multiplication is an intersection test. Both can be solved exactly once the embedded subfields are known.

For sums, normalized trace provides the further descent needed for unrestricted global optimization. For products, the article keeps the distinction between complete ambient-field pair optimization and bounded many-factor discovery explicit. In the two ninth-degree examples, that distinction does not leave any uncertainty about the answer: the requested cubic decompositions are recovered, and the universal degree obstruction proves that three is the smallest possible maximum degree, even with arbitrarily many components.

A Complete Wolfram Language implementation

The following listing is the same file distributed separately in the archive. It contains the coordinate layer, complete subfield construction, pair and additive optimizers, bounded product search, and inverse-resultant front end.

```
(* RootDecomposition.wl -- exact algorithms accompanying the article.
   Validation status: independently checked in SymPy; NOT executed in a
   Mathematica kernel in the authoring session. Run RunTests.wl locally.
   No numerical approximation is used to accept an identity.
*)
BeginPackage["RootDecomposition`"];
RootDegree::usage = "RootDegree[a] is the absolute algebraic degree over Q.";
RootDegreeLowerBound::usage = "RootDegreeLowerBound[a] gives a lower bound valid for any finite
     sum or product decomposition.";
RootSubfieldData::usage = "RootSubfieldData[theta] computes all embedded subfields of Q(theta),
     as rational row bases.";
RootPairDecompose::usage = "RootPairDecompose[a, \"Sum\"|\"Product\", data:Automatic] minimizes
     the maximum degree among TWO components in the supplied ambient field (default Q(a)).";
RootSumDecompose::usage = "RootSumDecompose[a, data:Automatic] optimizes arbitrary finite sums
    in the supplied ambient field (default Q(a)).";
RootGlobalSumDecompose::usage = "RootGlobalSumDecompose[a] optimizes arbitrary finite sums
    globally, using a normal closure when necessary. Potentially very expensive.";
RootPolynomialCatalogue::usage = "RootPolynomialCatalogue[d,H] lists all primitive irreducible
    integer polynomials of degrees 2 through d, coefficient height at most H and positive
    leading coefficient.";
RootProductFromCandidates::usage = "RootProductFromCandidates[a,candidates,d,R] exhausts
    products of at most R factors, with all but one factor from candidates; the last factor is
     tested exactly for degree <= d.";
BoundedRootProduct::usage = "BoundedRootProduct[a,d,H,R] searches products with at most R
    factors; all but one have minimal-polynomial height <= H and degree <= d. Failure to find
    is NOT global impossibility.";
```

```
RootPairFromPolynomial::usage = "RootPairFromPolynomial[a,p,x,op,d] uses a resultant to find a
    partner for a root of p; returns a verified pair or a scoped NotFound result.";
Begin["`Private`"];

$failureTag = Unique["RootDecompositionFailure"];
fail[tag_, detail_] := Throw[Failure[tag, <|"Detail" -> detail|>], $failureTag];
qQ[t_] := IntegerQ[t] || Head[t] === Rational;
zeroQ[t_] := TrueQ[RootReduce[t] === 0];
rr[t_] := Module[{v = Quiet[Check[RootReduce[t], $Failed]]},
  If[v === $Failed, fail["AlgebraicReduction", HoldForm[t]]]; v];

minimal[a_, x_] := Module[{p},
  If[!FreeQ[a, _Real], fail["InexactInput", "Use exact algebraic numbers, not decimal
    approximations."]];
  p = Quiet[Check[MinimalPolynomial[rr[a], x], $Failed]];
  If[p === $Failed || !PolynomialQ[p,x], fail["NotAlgebraic", HoldForm[a]]];
  If[!VectorQ[CoefficientList[p,x],qQ] || Exponent[p,x] < 1,
    fail["NotAlgebraicOverQ", HoldForm[a]]]; p];
degree[a_] := Module[{x}, Exponent[minimal[a,x],x]];
lower[a_] := Module[{n=degree[a]}, If[n==1,1,Max[First /@ FactorInteger[n]]]];
RootDegree[a_] := Catch[degree[a],$failureTag];
RootDegreeLowerBound[a_] := Catch[lower[a],$failureTag];

(* An integral primitive generator prevents ToNumberField from rescaling
   the generator behind our coordinate system. No integral BASIS is needed. *)
makeField[a_] := Module[{x,p,theta,n,c},
  p=minimal[a,x]; n=Exponent[p,x]; c=Coefficient[p,x,n];
  theta=rr[c a]; p=minimal[theta,x]; p=Expand[p/Coefficient[p,x,n]];
  <|"Theta"->theta,"Variable"->x,"Polynomial"->p,"Degree"->n|>];

vec[a_,ctx_] := Module[{n=ctx["Degree"],x=ctx["Variable"],v,an,p},
  v=rr[a]; If[qQ[v],Return[PadRight[{v},n]]];
  an=Quiet[Check[ToNumberField[v,ctx["Theta"]],$Failed]];
  If[Head[an]=!=AlgebraicNumber,fail["OutsideAmbientField",HoldForm[a]]];
  If[!zeroQ[an[[1]]-ctx["Theta"]],fail["GeneratorMismatch",an]];
  p=AlgebraicNumberPolynomial[an,x];
  p=PolynomialRemainder[p,ctx["Polynomial"],x];
  v=PadRight[CoefficientList[p,x],n];
  If[Length[v]!=n || !VectorQ[v,qQ],fail["BadCoordinates",v]]; v];
value[v_,ctx_] := rr[v.Table[ctx["Theta"]^j,{j,0,ctx["Degree"]-1}]];
mul[u_,v_,ctx_] := Module[{x=ctx["Variable"],n=ctx["Degree"],b,p},
  b=Table[x^j,{j,0,n-1}];
  p=PolynomialRemainder[Expand[(u.b)(v.b)],ctx["Polynomial"],x];
  PadRight[CoefficientList[p,x],n]];
rows[B_] := If[B===={}, {}, Select[RowReduce[B], !AllTrue[#,TrueQ[#==0]&]&]];
meet[A_,B_,n_] := Module[{eq=Join[NullSpace[A],NullSpace[B]]},
  If[eq==={},IdentityMatrix[n],rows[NullSpace[eq]]]];

(* A deterministic rational solution: free coordinates are set to zero.
   Missing is returned only after an exact inconsistency check. *)
qsolve[A_,b_] := Module[{R,n=Length[First[A]],z,p},
  R=RowReduce[MapThread[Append,{A,b}]];
  If[AnyTrue[R, AllTrue[Most[#],TrueQ[#==0]&] && Last[#]!=0 &],
    Return[Missing["Inconsistent"]]];
  z=ConstantArray[0,n];
  Do[p=Select[Range[n],row[[#]]!=0&,1];
    If[p=!={},z[[First[p]]]=Last[row]],{row,R}];
  If[A.z=!=b,fail["LinearSolveVerification",{A,b,z}]]; z];

(* For a factor g of minpoly(theta), compute
   {h(theta): g(X) divides h(X)-h(theta)} by rational linear algebra. *)
principal[g_,ctx_] := Module[{x=ctx["Variable"],n=ctx["Degree"],r,cols,h},
  r=Exponent[g,x];
```

```
  cols=Table[
    h=Expand[PolynomialRemainder[x^j,g,x]-ctx["Theta"]^j];
    Flatten[Table[vec[Coefficient[h,x,k],ctx],{k,0,r-1}]],
    {j,0,n-1}];
  rows[NullSpace[Transpose[cols]]]];

subfields[a_] := Module[{ctx=makeField[a],fl,gs,bs,new,B,n},
  n=ctx["Degree"];
  If[n==1,Return[<|"Field"->ctx,"Bases"->{IdentityMatrix[1]},
    "SubfieldDegrees"->{1},"FactorDegrees"->{1},"Complete"->True|>]];
  fl=Quiet[Check[FactorList[ctx["Polynomial"],Extension->{ctx["Theta"]}],$Failed]];
  If[fl===$Failed || !ListQ[fl] || Length[fl]<2,
    fail["ExtensionFactorizationFailed",ctx["Polynomial"]]];
  If[!AllTrue[Rest[fl],Last[#]==1&],fail["NonseparableFactorization",fl]];
  gs=First /@ Rest[fl]; bs={IdentityMatrix[n]};
  Do[B=principal[g,ctx]; new=meet[#,B,n]& /@ bs;
    bs=DeleteDuplicates[Join[bs,new]],{g,gs}];
  bs=SortBy[bs,Length];
  <|"Field"->ctx,"Bases"->bs,"SubfieldDegrees"->(Length /@ bs),
    "FactorDegrees"->(Exponent[#,ctx["Variable"]]& /@ gs),"Complete"->True|>];
RootSubfieldData[a_] := Catch[subfields[a],$failureTag];
getData[a_,data_] := Module[{z=If[data===Automatic,subfields[a],data]},
  If[!AssociationQ[z] || !TrueQ[z["Complete"]],
    fail["IncompleteSubfieldData","Supply completed RootSubfieldData output."]]; z];

normalizePair[terms_,op_] := Module[{b=terms[[1]],c=terms[[2]],x,p,m,mu,k,c0},
  p=minimal[b,x]; m=Exponent[p,x]; p=Expand[p/Coefficient[p,x,m]];
  If[op=="Sum",
    mu=-Coefficient[p,x,m-1]/m; Return[rr /@ {b-mu,c+mu}]];
  c0=Coefficient[p,x,0];
  If[c0==0,Return[terms]];
  k=rr[Surd[Abs[c0],m]];
  If[qQ[k] && k!=0,
    If[OddQ[m],k=Sign[c0] k]; rr /@ {b/k,c k}, terms]];

certificate[a_,terms_,op_,scope_,forceGlobal_:False] := Module[{ds,score,lb,ex},
  ds=degree /@ terms; score=Max[ds]; lb=lower[a];
  ex=If[op=="Sum",Total[terms],Times@@terms];
  If[!zeroQ[ex-a],fail["IdentityVerificationFailed",{a,terms,op}]];
  <|"Status"->"Success","Operation"->op,"Components"->terms,
    "Expression"->ex,"ComponentDegrees"->ds,"MaximumDegree"->score,
    "Verified"->True,"Scope"->scope,"GlobalDegreeLowerBound"->lb,
    "GlobalOptimalityProved"->TrueQ[forceGlobal || score==lb]|>];

pair[a_,op_,data_] := Module[{D,ctx,bs,av,pairs,F,G,w,ker,AG,u,v,terms},
  If[!MemberQ[{"Sum","Product"},op],fail["BadOperation",op]];
  If[degree[a]==1,Return[certificate[a,{rr[a]},op,"Global",True]]];
  D=getData[a,data]; ctx=D["Field"]; bs=D["Bases"]; av=vec[a,ctx];
  pairs=Flatten[Table[{i,j},{i,Length[bs]},{j,i,Length[bs]}],1];
  pairs=SortBy[pairs,Function[ij,{Max[Length /@ bs[[ij]]],Total[Length /@ bs[[ij]]]}]];
  Do[F=bs[[ij[[1]]]]; G=bs[[ij[[2]]]];
    If[op=="Sum",
      w=qsolve[Transpose[Join[F,G]],av];
      If[MissingQ[w],Continue[]];
      u=Take[w,Length[F]].F; v=Drop[w,Length[F]].G;
      terms=value[#,ctx]& /@ {u,v},
      AG=mul[av,#,ctx]& /@ G;
      ker=NullSpace[Transpose[Join[F,-AG]]];
      If[ker===={},Continue[]]; w=First[ker];
      u=Take[w,Length[F]].F; v=Drop[w,Length[F]].G;
      If[AllTrue[v,TrueQ[#==0]&],fail["UnexpectedZeroFactor",w]];
      terms={value[u,ctx],rr[1/value[v,ctx]]}];
    terms=normalizePair[terms,op];
```

```
    Return[certificate[a,terms,op,"OptimalTwoComponentsInAmbientField"]],
    {ij,pairs}];
  fail["InternalError","The trivial pair should always be available."]];
RootPairDecompose[a_,op_,data_:Automatic] := Catch[pair[a,op,data],$failureTag];

sumAny[a_,data_] := Module[{D,ctx,bs,av,levels,selected,B,w,parts,offset,len},
  If[degree[a]==1,Return[certificate[a,{rr[a]},"Sum","Global",True]]];
  D=getData[a,data]; ctx=D["Field"]; bs=D["Bases"]; av=vec[a,ctx];
  levels=Sort[DeleteDuplicates[Length /@ bs]];
  Do[selected=Select[bs,Length[#]<=d&]; B=Join@@selected;
    w=qsolve[Transpose[B],av]; If[MissingQ[w],Continue[]];
    offset=0; parts=Table[len=Length[F];
      With[{v=Take[w,{offset+1,offset+len}].F},offset+=len;value[v,ctx]],
      {F,selected}]; parts=Select[parts,!zeroQ[#]&];
    If[parts===={},parts={0}];
    Return[certificate[a,parts,"Sum","OptimalFiniteSumInAmbientField"]],
    {d,levels}];
  fail["InternalError","The whole ambient field should span the input."]];
RootSumDecompose[a_,data_:Automatic] := Catch[sumAny[a,data],$failureTag];

rootsOf[p_,x_] := Module[{fn=Function[Evaluate[p /. x->Slot[1]]]},
  Table[Root[fn,j],{j,Exponent[p,x]}]];
normalGenerator[a_] := Module[{x,p,all,reps,gens},
  p=minimal[a,x]; all=rootsOf[p,x];
  reps=Quiet[Check[ToNumberField[all,All],$Failed]];
  If[reps===$Failed || !ListQ[reps],fail["NormalClosureFailed",p]];
  gens=Cases[reps,AlgebraicNumber[t_,_List]:>t,{1}];
  If[gens==={},0,rr[First[gens]]]];
RootGlobalSumDecompose[a_] := Catch[Module[{ans,theta},
  ans=sumAny[a,Automatic];
  If[TrueQ[ans["GlobalOptimalityProved"]],Return[ans]];
  theta=normalGenerator[a]; ans=sumAny[a,subfields[theta]];
  Join[ans,<|"Scope"->"GlobalFiniteSumViaNormalClosure",
    "GlobalOptimalityProved"->True|>]],$failureTag];

catalogue[d_,H_,x_] := Module[{ans={},p},
  If[!IntegerQ[d] || d<1 || !IntegerQ[H] || H<1,
    fail["BadBounds",{d,H}]];
  Do[Do[If[GCD@@Join[cs,{lc}]!=1,Continue[]];
    p=lc x^m+cs.Table[x^j,{j,0,m-1}];
    If[TrueQ[IrreduciblePolynomialQ[p]],AppendTo[ans,p]],
    {lc,1,H},{cs,Tuples[Range[-H,H],m]}],{m,2,d}]; ans];
RootPolynomialCatalogue[d_,H_] := Catch[Module[{x},
  <|"Variable"->x,"Polynomials"->catalogue[d,H,x],
    "DegreeBound"->d,"CoefficientHeightBound"->H|>],$failureTag];

candidateProduct[a_,candidates_,d_,R_] := Module[{cat,walk,foundTag=Unique[],answer},
  If[!IntegerQ[d] || d<1 || !IntegerQ[R] || R<1,
    fail["BadBounds",{d,R}]];
  If[degree[a]<=d,Return[certificate[a,{rr[a]},"Product","CandidateSearch"]]];
  If[d<lower[a],Return[<|"Status"->"ImpossibleByDegreeLowerBound",
    "DegreeBound"->d,"GlobalDegreeLowerBound"->lower[a]|>]];
  cat=DeleteDuplicates[rr /@ candidates];
  If[!AllTrue[cat,!zeroQ[#] && degree[#]<=d&],
    fail["BadCandidates","Candidates must be nonzero and have absolute degree <= d."]];
  walk[prefix_,product_,start_,left_] := Module[{last,newproduct},
    last=rr[a/product];
    If[degree[last]<=d,
      Throw[certificate[a,Append[prefix,last],"Product","BoundedCandidateSearch"],foundTag]];
    If[left==0,Return[Null]];
    Do[newproduct=rr[product cat[[i]]];
      walk[Append[prefix,cat[[i]]],newproduct,i,left-1],{i,start,Length[cat]}]];
  answer=Catch[walk[{},1,1,R-1];Missing["NotFound"],foundTag];
```

```
  If[MissingQ[answer],<|"Status"->"NotFoundWithinBounds","DegreeBound"->d,
    "MaximumFactors"->R,"CandidateCount"->Length[cat],
    "GlobalImpossibilityProved"->False|>,answer]];
RootProductFromCandidates[a_,candidates_List,d_Integer,R_Integer] :=
  Catch[candidateProduct[a,candidates,d,R],$failureTag];
BoundedRootProduct[a_,d_Integer,H_Integer,R_Integer] := Catch[Module[{x,ps,cat,ans},
  If[d<1 || H<1 || R<1,fail["BadBounds",{d,H,R}]];
  If[degree[a]<=d,Return[certificate[a,{rr[a]},"Product","GlobalDegreeBound"]]];
  If[d<lower[a],Return[<|"Status"->"ImpossibleByDegreeLowerBound",
    "DegreeBound"->d,"GlobalDegreeLowerBound"->lower[a]|>]];
  ps=catalogue[d,H,x]; cat=Flatten[rootsOf[#,x]& /@ ps];
  ans=candidateProduct[a,cat,d,R];
  Join[ans,<|"CoefficientHeightBoundForAllButOneFactor"->H|>]],$failureTag];

(* An optional rational-resultant front end, not a rational-points solver. *)
RootPairFromPolynomial[a_,p_,x_Symbol,op_,d_Integer] := Catch[Module[
  {y,z,f,R,fl,qs,bs,cs},
  If[!MemberQ[{"Sum","Product"},op],fail["BadOperation",op]];
  If[d<1 || !PolynomialQ[p,x] || Exponent[p,x]<1 ||
    !VectorQ[CoefficientList[p,x],qQ],fail["BadCandidatePolynomial",p]];
  f=minimal[a,z]; bs=Select[rootsOf[p,x],degree[#]<=d&];
  If[op=="Product" && zeroQ[a],
    Return[If[bs===={},
      <|"Status"->"NotFoundForCandidatePolynomial",
        "GlobalImpossibilityProved"->False|>,
      certificate[a,{First[bs],0},op,"OneCandidatePolynomial"]]]];
  R=Expand[Resultant[p /. x->y,
    If[op=="Sum",f /. z->z+y,f /. z->z y],y]];
  If[R===0,fail["DegenerateResultant","Remove a zero root from the candidate polynomial."]];
  fl=FactorList[R]; qs=Select[First /@ Rest[fl],Exponent[#,z]<=d&];
  Do[cs=rootsOf[q,z];
    Do[If[op=="Product" && (zeroQ[b]||zeroQ[c]),Continue[]];
      If[zeroQ[If[op=="Sum",b+c,b c]-a],
        Return[certificate[a,normalizePair[{b,c},op],op,"OneCandidatePolynomial"]]],
      {b,bs},{c,cs}],{q,qs}];
  <|"Status"->"NotFoundForCandidatePolynomial","GlobalImpossibilityProved"->False|>
],$failureTag];

End[];
EndPackage[];
```

B Local Mathematica test suite

```
(* Run with: wolframscript -file code/RunTests.wl
   Or evaluate Get[".../code/RunTests.wl"] in Mathematica. *)
Get[FileNameJoin[{DirectoryName[$InputFileName],"RootDecomposition.wl"}]];
a=Root[1+#+#^3&,1]; b=Root[1-#+#^3&,1];
prod=Root[-1-#+3#^3-#^4+#^5-3#^6+2#^7+#^9&,1];
sum=Root[8-4#+24#^2-15#^3+3#^5+6#^6+#^9&,1];
prodData=RootSubfieldData[prod]; sumData=RootSubfieldData[sum];
prodAns=RootPairDecompose[prod,"Product",prodData];
sumAns=RootPairDecompose[sum,"Sum",sumData];
report=TestReport[{
 VerificationTest[RootReduce[prod-a b],0,TestID->"OriginalProduct"],
 VerificationTest[RootReduce[sum-a-b],0,TestID->"OriginalSum"],
 VerificationTest[RootDegree /@ {prod,sum},{9,9},TestID->"InputDegrees"],
 VerificationTest[prodData["SubfieldDegrees"],{1,3,3,9},TestID->"ProductSubfields"],
 VerificationTest[sumData["SubfieldDegrees"],{1,3,3,9},TestID->"SumSubfields"],
 VerificationTest[{prodAns["MaximumDegree"],prodAns["Verified"],
```

```
  prodAns["GlobalOptimalityProved"]},{3,True,True},TestID->"OptimalProduct"],
 VerificationTest[{sumAns["MaximumDegree"],sumAns["Verified"],
  sumAns["GlobalOptimalityProved"]},{3,True,True},TestID->"OptimalSum"],
 VerificationTest[RootSumDecompose[sum,sumData]["MaximumDegree"],3,
  TestID->"AnyLengthSum"],
 VerificationTest[RootPairFromPolynomial[prod,x^3+x+1,x,"Product",3]["MaximumDegree"],3,
  TestID->"ResultantProduct"],
 VerificationTest[RootPairFromPolynomial[sum,x^3+x+1,x,"Sum",3]["MaximumDegree"],3,
  TestID->"ResultantSum"],
 VerificationTest[RootSumDecompose[Sqrt[2]+Sqrt[3]+Sqrt[5]]["MaximumDegree"],2,
  TestID->"ThreeQuadraticSummands"],
 VerificationTest[RootPairDecompose[(Sqrt[2]+Sqrt[3])/2,"Sum"]["MaximumDegree"],2,
  TestID->"NonintegralInput"],
 VerificationTest[RootPairDecompose[0,"Product"]["Components"],{0},TestID->"Zero"],
 VerificationTest[RootPairFromPolynomial[0,x^2-2,x,"Product",2]["Verified"],True,
  TestID->"ZeroResultantProduct"],
 VerificationTest[FailureQ[BoundedRootProduct[1,2,1,0]],True,
  TestID->"RejectZeroFactorBound"],
 VerificationTest[RootPairDecompose[3/7,"Sum"]["MaximumDegree"],1,TestID->"Rational"],
 VerificationTest[FailureQ[RootDegree[1.25]],True,TestID->"RejectInexact"],
 VerificationTest[RootProductFromCandidates[(1+Sqrt[2])(1+Sqrt[3])(1+Sqrt[5]),
  {1+Sqrt[2],1+Sqrt[3],1+Sqrt[5]},2,3]["MaximumDegree"],2,
  TestID->"ThreeQuadraticFactors"]
}];
Print[report];
Print["Product: ",prodAns]; Print["Sum: ",sumAns];
```

References

[1] Vladimir Reshetnikov, *Factor a polynomial Root into Roots of smallest possible degree*, Mathematica Stack Exchange, question 105933; see also the constructive answer by yarchik and its discussion.
https://mathematica.stackexchange.com/questions/105933/

[2] Jonas Szutkoski and Mark van Hoeij, *The Complexity of Computing all Subfields of an Algebraic Number Field*, arXiv:1606.01140, version 3, November 20, 2017.
https://arxiv.org/abs/1606.01140
https://arxiv.org/html/1606.01140v3

[3] Wolfram Research, *Root*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/Root.html

[4] Wolfram Research, *RootReduce*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/RootReduce.html

[5] Wolfram Research, *MinimalPolynomial*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/MinimalPolynomial.html

[6] Wolfram Research, *ToNumberField*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/ToNumberField.html

[7] Wolfram Research, *AlgebraicNumber*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/AlgebraicNumber.html

[8] Wolfram Research, *AlgebraicNumberPolynomial*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html

[9] Wolfram Research, *Resultant*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/Resultant.html

[10] Wolfram Research, *PolynomialRemainder*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/PolynomialRemainder.html

[11] Wolfram Research, *FactorList*, Wolfram Language Documentation.
https://reference.wolfram.com/language/ref/FactorList.html

[12] SymPy Development Team, *Number Fields*, SymPy documentation.
https://docs.sympy.org/latest/modules/polys/numberfields.html

Public sources accessed September 6, 2026. All new computations and the explicitly given proofs are part of this report; the local Wolfram Language test suite is supplied but not represented as executed.